

İçindekiler

1. Giriş	3
Projenin Amacı	3
Problemin Tanımı	3
Hedef Kullanıcılar	3
Kullanılan Araç ve Teknolojiler	3
2. Proje Senaryosu	4
Kullanıcı Rollerini	4
Hangi İşlemleri Kim Yapabiliyor?	4
3. Gereksinim Analizi	5
Fonksiyonel Gereksinimler	5
Sistem Gereksinimleri	5
Yazılım Gereksinimleri:	5
Donanım Gereksinimleri (minimum):	5
Kullanıcı Arayüzü	5
4. UML Diyagramı	7
Kalıtım (Inheritance) İlişkileri	7
Association (İlişki) Bağlantıları	7
Sınıf Detayları	7
UML Diyagram Görseli	7
5. Yazılım Geliştirme Süreci	8
Kod Yapısı ve Katmanlar	8
Nesne Yönelimli Programlama (OOP) İlkeleri	8
Kod Kalitesi ve Yorumlar	9
7. Ekran Görüntüleri	10
1. LoginFrame – Giriş Ekranı	10
2. StudentDashboard – Öğrenci Paneli	10
3. TeacherDashboard – Öğretmen Paneli	10
4. AdminDashboard – Yönetici Paneli	11
9. Sonuç ve Değerlendirme	14
Kazanımlar	14
Genel Değerlendirme	14
Gelecek Geliştirme Fikirleri	14

1. Giriş

Bu proje, bir okul sisteminde öğrencilerin notlarının takibini kolaylaştırmak amacıyla geliştirilmiş “**Not Takip Sistemi**” adlı bir masaüstü uygulamasıdır. Uygulama, kullanıcıların rolüne göre farklı yetkilere sahip olmasını sağlayarak öğrenci, öğretmen ve yönetici (admin) girişlerini desteklemektedir. Sistem üzerinde kullanıcılar giriş yaptıktan sonra; öğrenciler kendi notlarını görüntüleyebilir, öğretmenler ders ataması yapılmış öğrencilerin vize ve final notlarını girebilir/güncelleyebilir, yöneticiler ise sistemde kullanıcıları ve dersleri yönetebilir.

Projenin Amacı

Projede amaç, bir eğitim kurumunun sınav notlarını dijital ortamda tutabileceği, kullanıcıların rollerine göre erişim sağlayabileceği ve not işlemlerini hatasız gerçekleştirebileceği bir sistem geliştirmektir. Bu sistem ile kâğıt veya Excel üzerinde yapılan takip işlemleri ortadan kaldırılarak, daha güvenli ve sürdürülebilir bir yapı oluşturulması hedeflenmiştir.

Problemin Tanımı

Günümüzde birçok eğitim kurumunda not takip işlemleri hâlâ manuel olarak ya da dağınık dijital belgeler aracılığıyla yapılmaktadır. Bu durum; veri bütünlüğü sorunlarına, erişim karmaşasına ve zaman kaybına neden olmaktadır. Ayrıca, kullanıcıların yetkisiz şekilde diğer kullanıcıların verilerine erişebilmesi güvenlik zafiyetine yol açmaktadır. Bu projede geliştirilen sistem, bu tür sorunları ortadan kaldırmak için kullanıcı tipine göre yetkilendirme, merkezi veri tabanı üzerinden işlem yapma ve kullanıcı dostu arayüzlerle etkileşim kurma özelliklerine sahiptir.

Hedef Kullanıcılar

Bu sistem, eğitim kurumlarında görev alan:

- Öğrenciler (kendi notlarını takip eden)
- Öğretmenler (ders verdikleri öğrencilerin notlarını giren ve güncelleyen)
- Yönetici (kullanıcı ve ders yönetimi yapan admin) tarafından kullanılmak üzere tasarlanmıştır.

Kullanılan Araç ve Teknolojiler

Projede aşağıdaki teknolojiler kullanılmıştır:

- **Java** – Uygulamanın tüm iş mantığı ve kullanıcı arayüzü Swing kütüphanesi ile Java dilinde geliştirilmiştir.
- **Swing** – Masaüstü uygulamasının grafik arayüzü bu kütüphane ile tasarlanmıştır.
- **JDBC (Java Database Connectivity)** – Java ile SQL Server veritabanı arasındaki bağlantı bu teknoloji aracılığıyla kurulmuştur.
- **SQL Server** – Tüm kullanıcı, ders, not gibi veriler ilişkisel veritabanında tutulmakta ve SQL komutları ile yönetilmektedir.

2. Proje Senaryosu

“Not Takip Sistemi” adlı bu masaüstü uygulama, eğitim kurumlarında öğrenci başarılarının takibini dijital ortamda yönetmek amacıyla geliştirilmiştir. Uygulama, farklı kullanıcı rollerine sahip kişilerin kendi yetki düzeylerinde sisteme erişmesini sağlar. Her kullanıcı tipi yalnızca kendi rolüne uygun işlemleri gerçekleştirebilir. Sistem sayesinde notların doğru, güvenli ve hızlı şekilde işlenmesi ve görüntülenmesi hedeflenmiştir.

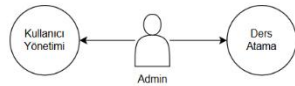
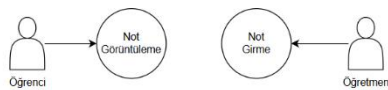
Uygulama açıldığında kullanıcı (öğrenci, öğretmen veya admin) kullanıcı adı ve şifresiyle giriş yapar. Doğrulama başarılıysa, ilgili kullanıcıya özel bir **dashboard (kontrol paneli)** açılır ve işlemlerini buradan gerçekleştirir.

Kullanıcı Roller

Kullanıcı Rolü	Yetkileri
Öğrenci	Kendi vize ve final notlarını görüntüleme
Öğretmen	Derslerine ait öğrencilere not girme ve güncelleme
Admin	Kullanıcı (öğrenci, öğretmen) ekleme, silme, güncelleme; ders oluşturma ve atama işlemleri

Hangi İşlemleri Kim Yapabiliyor?

- **Öğrenci:**
 - Giriş yaptıktan sonra yalnızca kendi derslerinin vize ve final notlarını görebilir.
 - Notlar sistemde varsa, ortalama ve başarı durumu da görüntülenir.
- **Öğretmen:**
 - Giriş yaptıktan sonra, sorumlu olduğu derslerdeki öğrencilerin listesine erişebilir.
 - İlgili öğrencilere ait vize ve final notlarını sisteme girebilir veya güncelleyebilir.
- **Admin:**
 - Tüm kullanıcıları yönetebilir (öğrenci, öğretmen).
 - Ders tanımlaması yapabilir.
 - Öğrencileri derslere atayabilir.
 - Ders öğretmen eşleştirmesi yapabilir.



3. Gereksinim Analizi

Gereksinim analizi, sistemin geliştirme sürecinde hangi işlevleri yerine getirmesi gerektiğini ve ne tür kaynaklara ihtiyaç duyduğunu belirlemek amacıyla yapılır. Bu bölümde projenin fonksiyonel gereksinimleri, sistem gereksinimleri ve kullanıcı arayüzü ile ilgili beklentiler ayrıntılı şekilde açıklanmıştır.

Fonksiyonel Gereksinimler

Sistemin temel işlevleri aşağıda listelenmiştir:

- **Giriş Sistemi:**
 - Kullanıcı adı ve şifre doğrulama
 - Rol bazlı (öğrenci, öğretmen, admin) oturum açma
- **Öğrenci İşlemleri:**
 - Kendi notlarını görüntüleyebilme (vize, final)
 - Ortalama not hesaplaması görüntüsü (opsiyonel)
- **Öğretmen İşlemleri:**
 - Derslerine ait öğrenci listelerini görüntüleyebilme
 - Öğrenciler için vize ve final notu girebilme
 - Not güncelleme işlemleri
- **Admin İşlemleri:**
 - Yeni kullanıcı (öğrenci, öğretmen, admin) ekleme
 - Kullanıcı güncelleme/silme işlemleri
 - Ders oluşturma ve yönetme
 - Öğrenci ve öğretmenlere ders atama

Sistem Gereksinimleri

Yazılım Gereksinimleri:

- Java Development Kit (JDK 17 veya üzeri)
- SQL Server 2019 (veya uyumlu sürüm)
- JDBC Driver (SQL Server için)
- Java Swing kütüphanesi
- Geliştirme ortamı: IntelliJ IDEA / Eclipse

Donanım Gereksinimleri (minimum):

- 4 GB RAM
- 1.5 GHz işlemci
- 500 MB boş disk alanı
- Windows 10 veya üzeri işletim sistemi (geliştirme için)

Kullanıcı Arayüzü

Uygulamanın arayüzü Java Swing ile geliştirilmiştir. Kullanıcı arayüzü sade, anlaşılır ve role göre özelleştirilmiş şekilde tasarlanmıştır.

- **Giriş Ekranı:**
 - Kullanıcı adı ve şifre giriş alanları
 - Giriş başarısız olursa uyarı mesajı
- **Öğrenci Paneli:**
 - Kayıtlı olduğu derslerin listesi
 - Her derse ait notların gösterimi
 - Ortalama ve başarı bilgisi (örneğin: Matematik - Vize: 75, Final: 80, Ortalama: 78
Durum: Başarılı)
- **Öğretmen Paneli:**
 - Ders listesinden seçim yapılabilen ComboBox
 - Öğrenci listesi için düzenlenebilir tablo (JTable)
 - Vize final sınavları girişi için textbox
- **Admin Paneli:**
 - Kullanıcı yönetimi (ekle/güncelle/sil)
 - Ders ekleme ve atama işlemleri için butonlar
 - Veri giriş formları ve geri bildirim mesajları

4. UML Diyagramı

Projenin temel yapısı, nesne yönelimli tasarım ilkelerine uygun şekilde UML sınıf diyagramı ile modellenmiştir. Aşağıdaki diyagramda sistemde yer alan tüm önemli sınıflar, kalıtım ilişkileri ve sınıflar arası bağlantılar detaylı şekilde gösterilmiştir.

Kalıtım (Inheritance) İlişkileri

- User sınıfı, sistemdeki temel soyut sınıf olarak tanımlanmıştır. login(), logout() ve openDashboard() gibi soyut metotlar bu sınıfta yer almakta ve alt sınıflar tarafından override edilmektedir.
- Student, Teacher ve Admin sınıfları, User sınıfından kalıtım yoluyla türetilmiştir.

Association (İlişki) Bağlantıları

- Course sınıfı:
 - Teacher sınıfı ile bire bir ilişkilidir. Her dersin bir öğretmeni vardır.
 - Student sınıfı ile bire-çok ilişkilidir. Her ders, birden fazla öğrenciye atanabilir.
- Student sınıfı:
 - Grade sınıfı ile bire-çok ilişkilidir. Her öğrencinin birden fazla dersten aldığı not bilgisi bulunmaktadır.

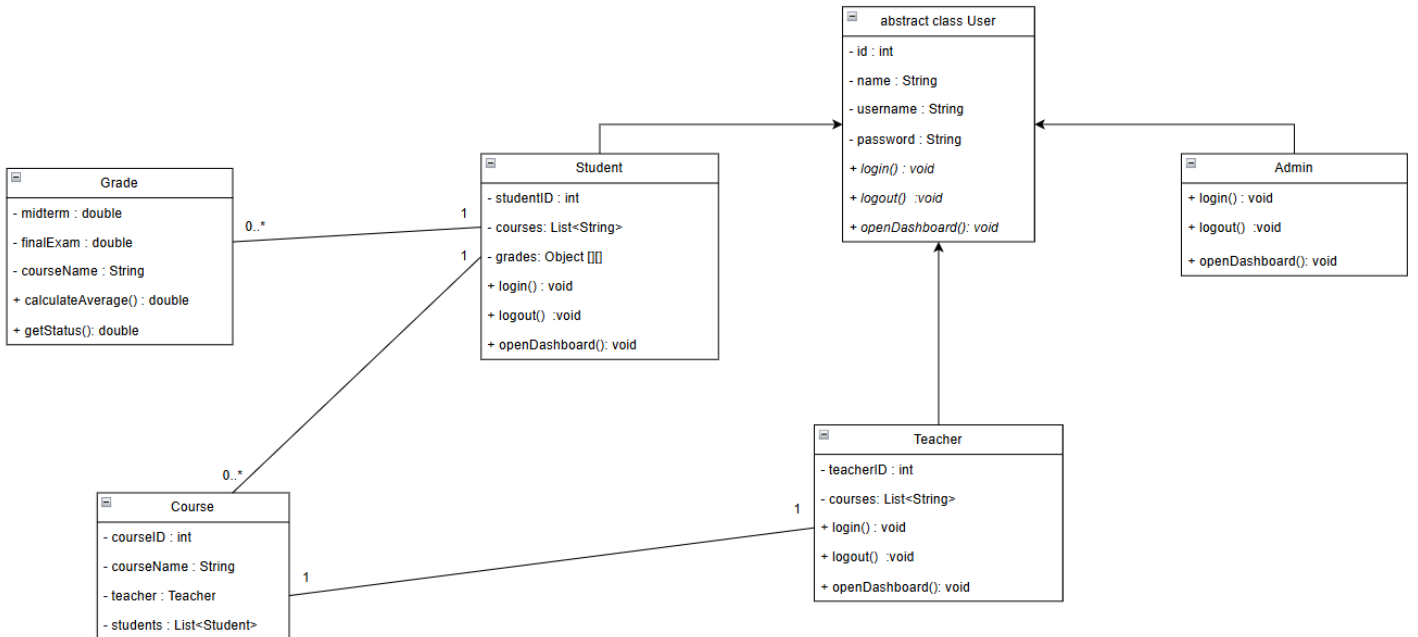
Sınıf Detayları

Her sınıf içinde;

- Veri elemanları (özellikler) ve bu elemanların veri tipleri (int, String, List<...> gibi)
- Ana işlevi oluşturan metotlar (örneğin viewGrades(), enterGrades(), calculateAverage() gibi)
- Soyut metotlar (örneğin openDashboard()), abstract olarak tanımlanmış ve alt sınıflarda override edilmiştir.

UML Diyagram Görseli

Projeye ait sınıflar ve ilişkiler aşağıdaki UML sınıf diyagramı ile görsel olarak sunulmuştur:



5. Yazılım Geliştirme Süreci

Bu bölümde, projenin geliştirilme sürecinde izlenen mimari yapı, kod organizasyonu ve kullanılan nesne yönelimli programlama (OOP) ilkeleri açıklanmıştır. Proje katmanlı bir yapıda modellenmiş olup, veri erişimi, iş mantığı ve kullanıcı arayüzü bileşenleri birbirinden ayrılmıştır. Bu yaklaşım kodun okunabilirliğini ve sürdürülebilirliğini artırmıştır.

Kod Yapısı ve Katmanlar

Projede temel olarak şu katmanlar kullanılmıştır:

1. Model (Veri Nesneleri / Entity) Katmanı:

- User, Student, Teacher, Admin, Course, Grade sınıfları bu katmanda yer alır.
- Bu sınıflar, sistemde kullanılan temel veri yapılarıdır.

2. DAO (Data Access Object) Katmanı:

- StudentDAO, TeacherDAO, AdminDAO, UserDAO gibi sınıflar bulunur.
- Bu sınıflar, veritabanı ile etkileşimi sağlar (SQL sorguları içerir).
- Örneğin createStudent() metodu, öğrenci verisini hem Users hem Students tablolarına ekler.

3. Service (İş Mantığı) Katmanı:

- StudentService gibi sınıflar yer alır.
- DAO katmanı ile arayüz katmanı arasında bir köprü görevi görür.
- İş kuralları burada yazılır (örneğin öğrenci ID hesaplaması, kullanıcı adı kontrolü).

4. UI (Kullanıcı Arayüzü) Katmanı:

- Swing tabanlı arayüzler (LoginFrame, StudentDashboard, AddTeacherFrame vs.)
- Kullanıcı girişleri, buton işlemleri ve tablolar burada yönetilir.

Nesne Yönelimli Programlama (OOP) İlkeleri

Proje geliştirilirken temel nesne yönelimli programlama ilkeleri uygulanmıştır:

• Encapsulation (Kapsülleme):

- Tüm sınıflarda alanlar private olarak tanımlanmış, dış erişim getter ve setter metotları ile sağlanmıştır.
- Bu sayede veri güvenliği ve kontrolü sağlanmıştır.

• Inheritance (Kalıtım):

- User soyut sınıfı, Student, Teacher ve Admin sınıfları tarafından kalıtılmıştır.
- Ortak metotlar (login(), logout(), openDashboard()) User sınıfında tanımlanmış ve gerektiğinde override edilmiştir.

• Polymorphism (Çok Biçimlilik):

- openDashboard() metodu her alt sınıfta farklı şekilde uygulanmıştır.

- Bu sayede her kullanıcı kendi paneline yönlendirilir.
- `User user = new Student(...)` şeklinde üst sınıf referansı ile alt sınıf nesnesi kullanılabilir.
- **Abstraction (Soyutlama):**
 - User sınıfı abstract olarak tanımlanmış ve somutlaştırılmamıştır.
 - Bu sınıfta yalnızca tüm kullanıcıların ortak davranışları tanımlanmıştır.

Kod Kalitesi ve Yorumlar

Kodlama sırasında aşağıdaki prensiplere dikkat edilmiştir:

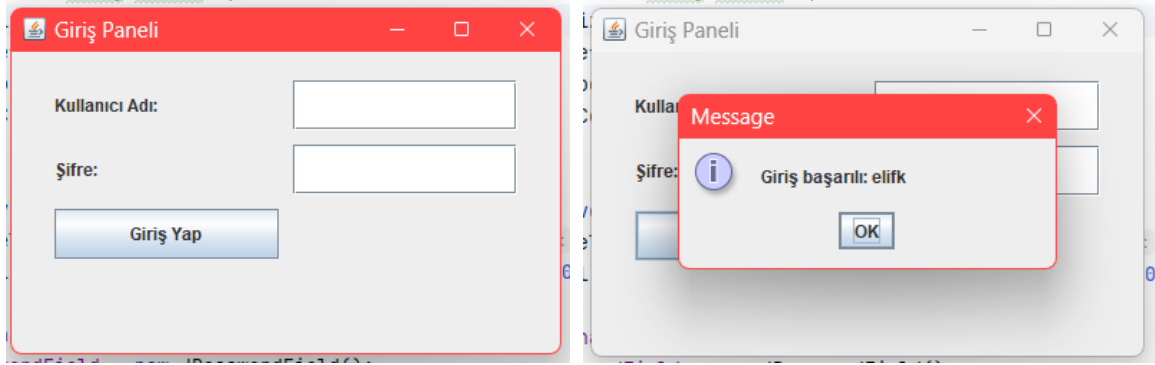
- Yorum satırları ile açıklamalar yapılmıştır.
- Değişken ve sınıf isimlendirmeleri anlamlı ve okunaklıdır.
- Test sınıfları (StudentDAOTest, TeacherDAOTest, AdminDAOTest) ile işlevlerin doğruluğu kontrol edilmiştir.

7. Ekran Görüntüleri

Aşağıdaki ekran görüntüleri, Not Takip Sistemi projesinin arayüzlerini ve temel işlevlerini göstermektedir:

1. LoginFrame – Giriş Ekranı

- Kullanıcı adı ve şifre giriş alanları



Şekil 1 Giriş Paneli

2. StudentDashboard – Öğrenci Paneli

- Öğrenci giriş yaptıktan sonra açılır
- Kayıtlı derslerin ve notların görüntülediği tablo



Ders Adı	Vize	Final	Ortalama	Durum
Edebiyat	70.0	80.0	76.0	Başarılı

Şekil 2 Öğrenci Paneli

3. TeacherDashboard – Öğretmen Paneli

- Öğretmenin ders listesinin olduğu ComboBox
- Seçilen derse ait öğrencilerin listelendiği tablo (JTable)

- Not girme/güncelleme alanı (textbox)

Öğretmen Paneli - Lütfiye Atar (Öğretmen No: 2005)

Ders Listesi: Edebiyat Dersi Seç

Seçili öğrenci: -

Not Girişi:

Vize Notu:

Final Notu:

Notu Kaydet

Öğrenci No	Ad Soyad	Kullanıcı Adı	Vize	Final	Ortalama	Durum
1002	Elif Kılıç	elifk	-	-	-	-

Seçilen Ders: Edebiyat

Şekil 3 Öğretmen Paneli

Öğretmen Paneli - Lütfiye Atar (Öğretmen No: 2005)

Ders Listesi: Edebiyat Dersi Seç

Seçili öğrenci: Elif Kılıç

Not Girişi:

Vize Notu:

Final Notu:

Notu Kaydet

ID	Öğrenci No	Ad Soyad	Kullanıcı Adı	Vize	Final	Ortalama	Durum
21	1002	Elif Kılıç	elifk	70.0	80.0	76.00	Başarılı

Seçilen Ders: Edebiyat

Şekil 4 Öğretmenin combobox'tan ders seçip o dersin öğrencilerini listelemesi

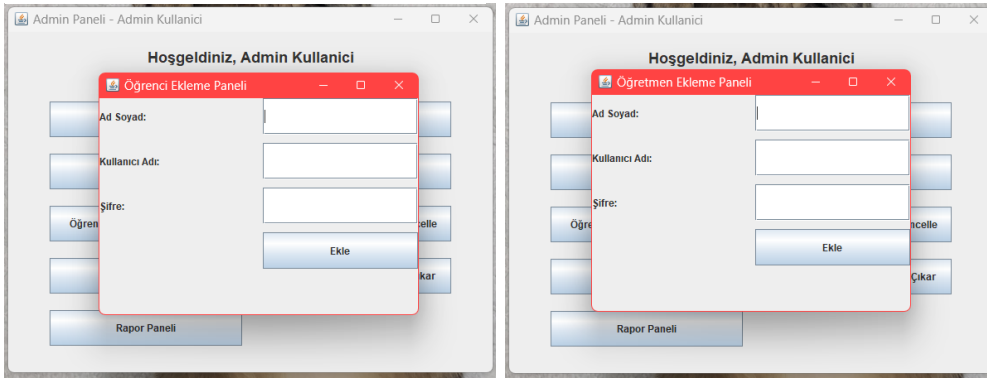
4. AdminDashboard – Yönetici Paneli

- Kullanıcı (öğrenci/öğretmen) ekleme, silme, güncelleme butonları
- Ders oluşturma ve ders atama ekranları
- Dersin öğrencilerini listeleme ekranı

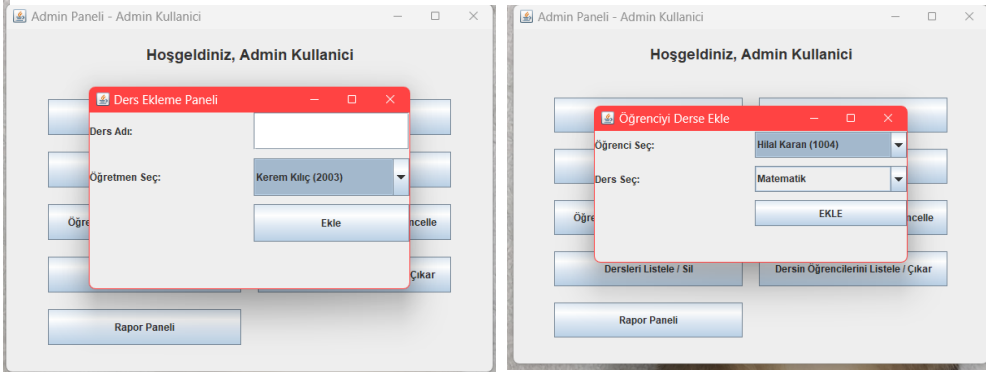
- Rapor paneli



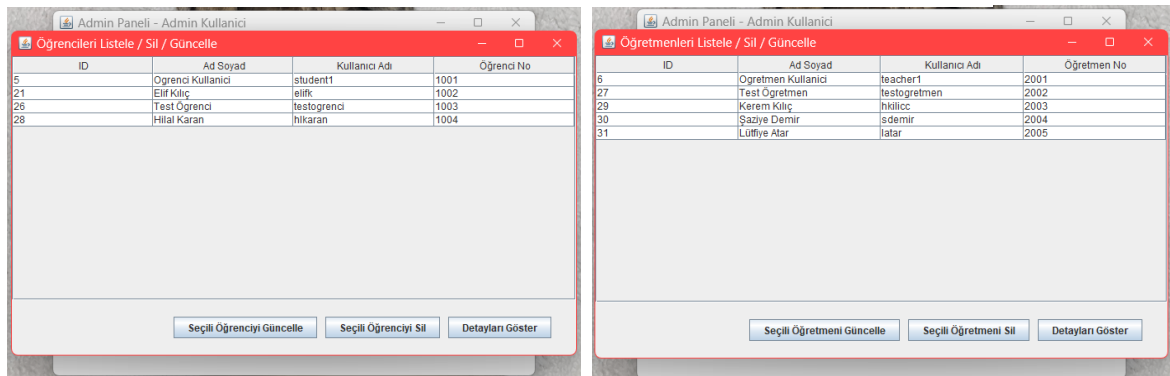
Şekil 8 Admin Paneli



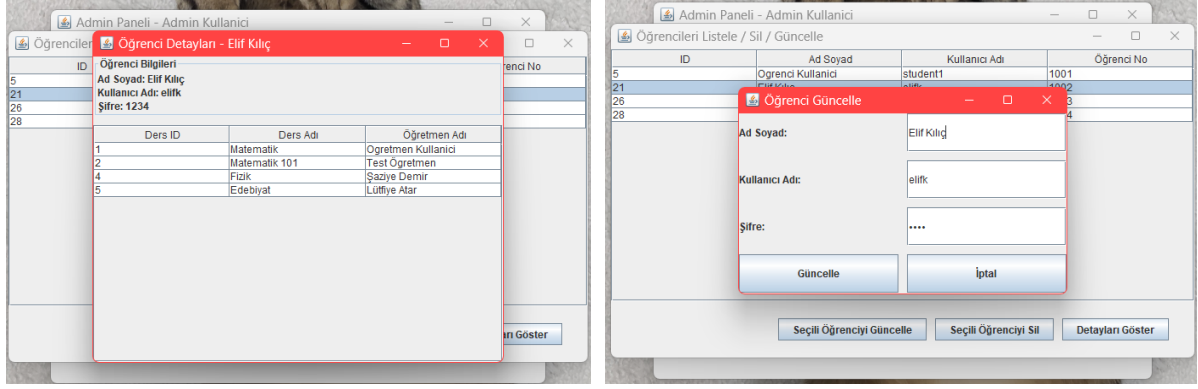
Şekil 7 Adminin öğrenci ve öğretmen ekleme arayüzü



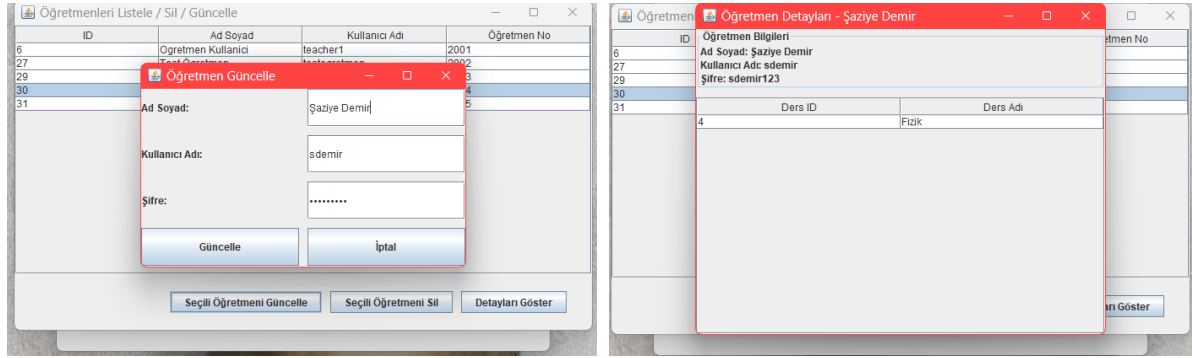
Şekil 6 Adminin ders ekleme ve öğrenciyi derse ekleme arayüzü



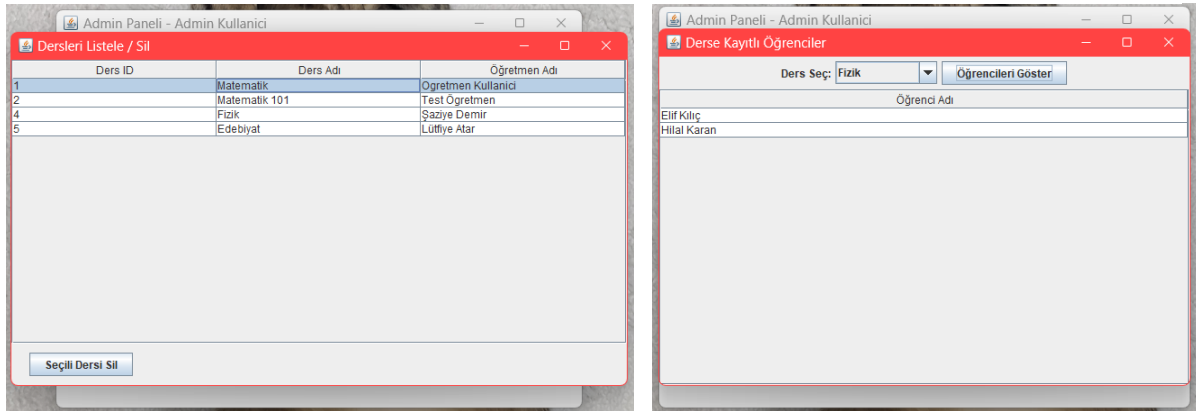
Şekil 5 Adminin öğrenci ve öğretmen listeleyebildiği ve güncelleyebildiği arayüz



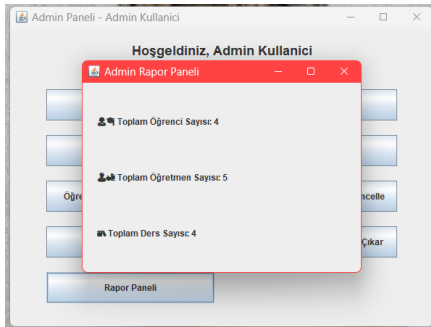
Şekil 12 Liste/sil/güncelle arayüzünde 'Seçili Öğrenciyi Güncelle' ve 'Detayları Göster' butonuna basıldığında açılan arayüz



Şekil 11 Liste/sil/güncelle arayüzünde 'Seçili Öğretmeni Güncelle' ve 'Detayları Göster' butonuna basıldığında açılan arayüz



Şekil 10 Dersleri Liste/Sil butonuna basıldığında açılan arayüz -----Dersin Öğrencilerini Liste butonuna basıldığında açılan arayüz



Şekil 9 Rapor Paneli butonuna basıldığında açılan arayüz

9. Sonuç ve Değerlendirme

Bu proje kapsamında geliştirilen **Not Takip Sistemi**, bir eğitim kurumunda kullanıcıların rollerine göre giriş yapabildiği, not işlemlerini yönetebildiği, basit ve işlevsel bir masaüstü uygulama olarak başarıyla tamamlanmıştır. Kullanıcı yönetimi, ders atamaları ve not işlemleri gibi temel işlevler eksiksiz şekilde çalışmaktadır.

Kazanımlar

- **Java programlama dili** kullanılarak Swing kütüphanesi ile kullanıcı arayüzü geliştirme deneyimi kazanılmıştır.
- **Katmanlı mimari yapısı** (Model-DAO-Service-UI) uygulanarak yazılım geliştirme süreçlerinde kodun modüler ve sürdürülebilir olması sağlanmıştır.
- **Veritabanı bağlantısı (JDBC)** kullanılarak SQL Server ile veri etkileşimi kurulmuş, SQL sorguları ile CRUD işlemleri gerçekleştirilmiştir.
- **Nesne Yönelimli Programlama (OOP)** ilkeleri (kalıtım, çok biçimlilik, kapsülleme) etkin bir şekilde kullanılmıştır.
- Sınıf diyagramları ve kullanım senaryosu diyagramı gibi **UML araçlarıyla yazılım tasarımı** yapılmıştır.

Genel Değerlendirme

Proje hem teknik becerilerin hem de yazılım geliştirme süreçlerinin pratiğe döküldüğü bir uygulama olmuştur. NYP ilkelerini teorik olarak bilsem de uygulama geliştirme aşamasında kullanmak bu ilkeleri pekiştirmemi ve daha iyi öğrenmemi sağlamıştır. Veritabanı bağlantısı ve SQL sorguları başlarda beni zorlasa da sonrasında bu konuda gelişimime destek olmuştur. Ayrıca ilk defa katmanlı mimari yapısını öğrenip uygulamak da bana çok şey katmıştır.

Ayrıca uygulama şu anda bilgi kısıtlılıklarım da sebebiyle sadece lokal ortamda çalışmaktadır. Ek olarak backend tarafında kodlarım eksiksiz bulunsa da Swing kütüphanesiyle kullanıcı arayüzünü geliştirme sürecinde bileşenlerin yerleşimi ve düzenlenmesi konusunda sorun yaşadığımdan ötürü arayüzlerim sade ve çoğunlukla tek görevi yerine getiren arayüzlerden oluşmaktadır. Bu durum arayüz ve buton sayısının artmasına sebep olmuş ve kullanıcı deneyimini olumsuz etkilemiştir.

Gelecek Geliştirme Fikirleri

- Mevcut arayüz konusunda admin panelinde öğretmen ve öğrenci işleri başlangıçta birbirinden ayrılabilir ve açılan pencerede birden çok görevi gerçekleştirecek şekilde örneğin sekmeli arayüz (JTabbedPane) kullanarak buton olayları azaltılabilir. Bu sayede daha compact kullanıcı dostu bir uygulama olur.
- Daha fazla özellik eklenerek (Örneğin öğrencinin ders seçme işlemi, admin için sistemde daha fazla kontrol verilmesi, öğretmenin öğrencilerle ilgili işlemlerinin artması, sınıf sistemi getirilmesi ya da dönem olabilir) uygulamanın daha etkin kullanımının sağlanması.
- Sistemin çevrim içi kullanılabilir hale gelmesi sağlanabilir.
- Web tabanlı bir arayüz tercih edilerek hem daha modern bir kullanıcı deneyimi hem de platformdan bağımsız erişim (mobil, tablet, tarayıcı vs.) sağlanabilir. Direkt mobil uygulama (Android Studio - Java/Kotlin) olarak da tasarlanabilir.